NODE.JS + EXPRESS + MONGODB

Student Management REST API

Complete Step-by-Step Guide

What to do Where to do it Why When

Simple explanation + complete colorful code

CREATE READ UPDATE DELETE

Simple folder structure 5 CRUD endpoints No custom middleware

Roadmap: what we do, in order

Follow the steps from top to bottom. Every step tells you **what** to do, **where** to do it, **why** it is needed and **when** to do it. Do not skip steps, each one depends on the one before it.

Step	Task	File / Command
1	Create the project folder and run npm init	`terminal`
2	Install the packages	`npm install`
3	Create the folder structure	`src/ + 5 folders`
4	Add environment variables	`.env, .gitignore`
5	Connect to MongoDB	`src/config/db.js`
6	Define the Student model	`src/models/student.model.js`
7	Write the CRUD logic	`src/controllers/student.controller.js`
8	Map URLs to the logic	`src/routes/student.routes.js`
9	Create the Express app	`src/app.js`
10	Create the server entry point	`src/server.js`
11	Add run scripts	`package.json`
12	Start MongoDB and run the API	`npm run dev`
13	Test all 5 CRUD APIs	`curl / Postman`

ABOUT "NO MIDDLEWARE"
We use **no custom middleware** (no auth, logger, or error handler files). The only thing we switch on is **express.json()**, which is built into Express and is required so the server can read JSON from the request body. Errors are handled inside each controller with try/catch.

1. What we are building

A simple **Student Management backend**. It is a REST API: a program that waits for HTTP requests (like POST or GET) and answers with JSON data. Student information is stored in a **MongoDB** database. We use **Node.js** to run JavaScript on the server, **Express** to create the routes, and **Mongoose** to talk to MongoDB easily.

CRUD means the 4 basic things you can do with data:

Letter	Meaning	HTTP method	Example URL
C	Create a student	`POST`	`/api/students`
R	Read all students	`GET`	`/api/students`
R	Read one student	`GET`	`/api/students/:id`
U	Update a student	`PUT`	`/api/students/:id`
D	Delete a student	`DELETE`	`/api/students/:id`

How one request travels



The client sends a request. The **route** decides which function must handle it. The **controller** function runs the logic. It uses the **model** to read or write data, and the model talks to **MongoDB**. The answer travels back the same way as JSON.

2. What you need before starting

Tool	Why	Check with
Node.js 18 or newer	Runs JavaScript on the server	`node -v`
MongoDB (local) or MongoDB Atlas (free cloud)	Stores the students	`mongosh --version`
VS Code (or any editor)	To write the code	-
Postman or curl	To test the APIs	`curl --version`

> **TIP**
> If you do not want to install MongoDB, create a free cluster on **MongoDB Atlas**, copy its connection string, and paste it into MONGO_URI in Step 4. Everything else stays the same.

3. Folder structure (and why)

project structure

```
student-management/
|-- src/
|   |-- config/
|   |   `-- db.js                    <- MongoDB connection
|   |-- models/
|   |   `-- student.model.js         <- Student schema
|   |-- controllers/
|   |   `-- student.controller.js    <- CRUD logic
|   |-- routes/
|   |   `-- student.routes.js        <- URL to function mapping
|   |-- app.js                       <- Express app setup
|   `-- server.js                    <- starts everything
|-- .env                             <- secrets and settings
|-- .gitignore
`-- package.json
```

Folder / File	Responsibility (one job each)
config/	Settings and connections, here only the database connection
models/	Describes what a Student looks like and talks to the collection
controllers/	The real work: create, read, update, delete logic
routes/	Says which URL + method calls which controller function
app.js	Builds the Express app and plugs the routes in
server.js	Loads .env, connects the database, starts listening on a port

Why split into folders? When each file has one job, you always know where to look. If a field is wrong you open the model. If a URL is wrong you open the routes. This is how real projects stay clean as they grow.

STEP 1 Create the project

WHAT	Make a new folder and run **npm init -y** to create package.json.
WHERE	In your terminal (Command Prompt, PowerShell, or VS Code terminal), in the place where you keep projects.
WHY	package.json stores the project name, scripts, and the list of packages. Node needs it to manage dependencies.
WHEN	Always first, before anything else.

Terminal

```
mkdir student-management
cd student-management
npm init -y
code .          # optional: opens the folder in VS Code
```

After **npm init -y** a file called package.json appears. We will edit it in Step 11.

STEP 2 Install the packages

WHAT	Install the 3 packages the app needs, plus nodemon for development.
WHERE	Terminal, inside the **student-management** folder.
WHY	**express** creates the server and routes. **mongoose** connects to MongoDB and gives us models. **dotenv** reads the .env file. **nodemon** restarts the server when you save a file.
WHEN	Right after Step 1, before writing any code.

Terminal

```
npm install express mongoose dotenv
npm install --save-dev nodemon
```

Package	Type	Used for
express	dependency	Web server and routing
mongoose	dependency	MongoDB connection, schema, queries
dotenv	dependency	Loads values from .env into process.env
nodemon	devDependency	Auto restart while developing

STEP 3 Create the folders and files

WHAT	Create the **src** folder with config, models, controllers, and routes inside it. Then create empty files.
WHERE	Inside the project root. You can create them in VS Code or with the commands below.
WHY	Having the structure ready first means you only paste code into the correct files afterwards.
WHEN	After installing packages, before writing code.

Terminal

```
# Mac / Linux / Git Bash
mkdir -p src/config src/models src/controllers src/routes
touch src/app.js src/server.js
touch src/config/db.js src/models/student.model.js
touch src/controllers/student.controller.js src/routes/student.routes.js
touch .env .gitignore

# Windows PowerShell
mkdir src\config, src\models, src\controllers, src\routes
ni src\app.js, src\server.js, .env, .gitignore
ni src\config\db.js, src\models\student.model.js
ni src\controllers\student.controller.js, src\routes\student.routes.js
```

STEP 4 Environment variables

WHAT	Put the port and database URL in **.env**. Tell Git to ignore .env and node_modules in **.gitignore**.
WHERE	Two files in the project root (same level as package.json).
WHY	The database URL can contain a password. Keeping it in .env means it is not hard-coded in the code and is not uploaded to GitHub.
WHEN	Before writing db.js, because db.js and server.js read these values.

.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/student_management
```

.gitignore

```
node_modules
.env
```

student_management at the end of MONGO_URI is the database name. MongoDB creates it automatically when the first student is saved. For Atlas, replace the whole URL with your Atlas connection string.

STEP 5 Connect to MongoDB

WHAT	Write a function **connectDB** that connects Mongoose to MongoDB.
WHERE	**src/config/db.js**
WHY	The app needs the database before it can save anything. Keeping the connection in its own file lets us reuse and change it in one place.
WHEN	After .env is ready. This is the first code file.

src/config/db.js

```
1  const mongoose = require("mongoose");
2
3  // Connects our app to MongoDB using the URL stored in .env
4  const connectDB = async () => {
5    try {
6      const conn = await mongoose.connect(process.env.MONGO_URI);
7      console.log("MongoDB connected: " + conn.connection.host);
8    } catch (error) {
9      console.error("MongoDB connection failed: " + error.message);
10     process.exit(1); // stop the app, it cannot work without a database
11   }
12 };
13
14 module.exports = connectDB;
```

- **async / await**: connecting takes time, so we wait for it to finish.
- **process.env.MONGO_URI**: the value from the .env file.
- **process.exit(1)**: if the database is down, stop the app and show the error instead of running half broken.
- **module.exports**: lets other files use this function with require().

STEP 6 Create the Student model

WHAT	Describe a student with a Mongoose **schema** and create the **Student** model from it.
WHERE	**src/models/student.model.js**
WHY	MongoDB has no fixed structure. The schema adds rules (required, unique, min, max) so only clean data is saved.
WHEN	After the DB connection, before the controller (the controller imports this model).

src/models/student.model.js

```
const mongoose = require("mongoose");

// Schema = the shape of one student document in MongoDB
const studentSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true,
    },
    email: {
      type: String,
      required: [true, "Email is required"],
      unique: true,        // no two students can share an email
      lowercase: true,
      trim: true,
    },
    age: {
      type: Number,
      min: [3, "Age must be at least 3"],
      max: [100, "Age must be at most 100"],
    },
    course: {
      type: String,
      required: [true, "Course is required"],
      trim: true,
    },
    phone: {
      type: String,
      trim: true,
    },
  },
  { timestamps: true }      // adds createdAt and updatedAt automatically
);

// Model = the tool we use to read/write the "students" collection
module.exports = mongoose.model("Student", studentSchema);
```

Field	Rule	Meaning
name	required, trim	Must be given. Extra spaces at start and end are removed.
email	required, unique, lowercase	Must be given. Two students cannot have the same email.
age	min 3, max 100	Optional number in this range.
course	required	Must be given.
phone	trim	Optional text.

Field	Rule	Meaning
`timestamps`	true	Adds createdAt and updatedAt automatically.

The model name **"Student"** becomes the collection **students** in MongoDB (lowercase and plural).

STEP 7 Write the controller (CRUD logic)

WHAT	Write 5 functions: createStudent, getAllStudents, getStudentById, updateStudent, deleteStudent.
WHERE	**src/controllers/student.controller.js**
WHY	This is the brain of the API. Each function reads the request, talks to the database through the model, and sends back a JSON response with the right status code.
WHEN	After the model is done. Routes (next step) need these functions.

src/controllers/student.controller.js

```
const mongoose = require("mongoose");
const Student = require("../models/student.model");

// ---------- helper: one place to turn errors into responses ----------
const handleError = (res, error) => {
  if (error.name === "ValidationError") {
    const messages = Object.values(error.errors).map((e) => e.message);
    return res.status(400).json({ success: false, message: messages.join(", ") });
  }
  if (error.code === 11000) {
    return res.status(409).json({ success: false, message: "Email already exists" });
  }
  return res.status(500).json({ success: false, message: error.message });
};

// Only these fields are allowed to be saved / updated
const pickFields = (body = {}) => {
  const allowed = ["name", "email", "age", "course", "phone"];
  const data = {};
  allowed.forEach((key) => {
    if (body[key] !== undefined) data[key] = body[key];
  });
  return data;
};

// ---------- CREATE : POST /api/students ----------
const createStudent = async (req, res) => {
  try {
    const student = await Student.create(pickFields(req.body));
    res.status(201).json({ success: true, message: "Student created", data: student });
  } catch (error) {
    handleError(res, error);
  }
};

// ---------- READ ALL : GET /api/students ----------
const getAllStudents = async (req, res) => {
  try {
    const students = await Student.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: students.length, data: students });
  } catch (error) {
    handleError(res, error);
  }
};

// ---------- READ ONE : GET /api/students/:id ----------
const getStudentById = async (req, res) => {
  try {
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({ success: false, message: "Invalid student id" });
    }

    const student = await Student.findById(id);
    if (!student) {
      return res.status(404).json({ success: false, message: "Student not found" });
    }
    res.status(200).json({ success: true, data: student });
  } catch (error) {
    handleError(res, error);
  }
};

// ---------- UPDATE : PUT /api/students/:id ----------
const updateStudent = async (req, res) => {
  try {
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
```

```
      return res.status(400).json({ success: false, message: "Invalid student id" });
    }

    const student = await Student.findByIdAndUpdate(id, pickFields(req.body), {
      new: true,            // return the updated document
      runValidators: true,  // apply schema rules on update too
    });
    if (!student) {
      return res.status(404).json({ success: false, message: "Student not found" });
    }
    res.status(200).json({ success: true, message: "Student updated", data: student });
  } catch (error) {
    handleError(res, error);
  }
};

// ---------- DELETE : DELETE /api/students/:id ----------
const deleteStudent = async (req, res) => {
  try {
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({ success: false, message: "Invalid student id" });
    }

    const student = await Student.findByIdAndDelete(id);
    if (!student) {
      return res.status(404).json({ success: false, message: "Student not found" });
    }
    res.status(200).json({ success: true, message: "Student deleted" });
  } catch (error) {
    handleError(res, error);
  }
};

module.exports = {
  createStudent,
  getAllStudents,
  getStudentById,
  updateStudent,
  deleteStudent,
};
```

How to read the controller

- **handleError**: one helper that turns errors into clear responses. Validation error becomes 400, duplicate email becomes 409, anything else becomes 500.
- **pickFields**: copies only name, email, age, course, phone from the body. This blocks anyone from sending unwanted fields.
- **createStudent**: `Student.create()` saves a new document and returns it with status 201.
- **getAllStudents**: `Student.find()` gets all; `sort({ createdAt: -1 })` shows the newest first.
- **getStudentById**: checks the id format first (`isValidObjectId`), then `findById()`. If nothing is found, it sends 404.
- **updateStudent**: `findByIdAndUpdate()`. **new: true** returns the updated student; **runValidators: true** keeps the schema rules active during update.
- **deleteStudent**: `findByIdAndDelete()` removes the student, or sends 404 if not found.

STEP 8 Create the routes

WHAT	Connect each URL + HTTP method to a controller function.
WHERE	**src/routes/student.routes.js**
WHY	Routes are the map of your API. Keeping them separate shows all endpoints in one glance.
WHEN	After the controller, because routes import its functions.

src/routes/student.routes.js

```
const express = require("express");
const {
  createStudent,
  getAllStudents,
  getStudentById,
  updateStudent,
  deleteStudent,
} = require("../controllers/student.controller");

const router = express.Router();

// /api/students
router.post("/", createStudent);
router.get("/", getAllStudents);

// /api/students/:id
router.get("/:id", getStudentById);
router.put("/:id", updateStudent);
router.delete("/:id", deleteStudent);

module.exports = router;
```

The base path **/api/students** is added in app.js (next step). So `router.get("/:id")` here becomes `GET /api/students/:id`. The **:id** part is a variable that arrives as `req.params.id`.

STEP 9 Create the Express app

WHAT	Create the Express app, turn on JSON reading, and attach the student routes.
WHERE	**src/app.js**
WHY	Without **express.json()**, req.body is empty and POST/PUT would not work. The app is exported (not started here) so the file stays simple.
WHEN	After the routes are ready.

src/app.js

```
const express = require("express");
const studentRoutes = require("./routes/student.routes");

const app = express();

// Built-in Express feature that lets us read JSON sent in req.body
// (this is NOT a custom middleware, it is required for any JSON API)
app.use(express.json());

// Health check
app.get("/", (req, res) => {
  res.json({ success: true, message: "Student Management API is running" });
});

// All student APIs start with /api/students
app.use("/api/students", studentRoutes);

module.exports = app;
```

STEP 10 Create the server entry point

WHAT	Load .env, connect to MongoDB, and then start listening on the port.
WHERE	**src/server.js**
WHY	This is the file Node runs first. It starts the whole app in the right order: environment, database, then server.
WHEN	Last code file. Everything else must exist before this runs.

src/server.js

```
require("dotenv").config();            // load .env FIRST

const app = require("./app");
const connectDB = require("./config/db");

const PORT = process.env.PORT || 5000;

// Start the server only after MongoDB is connected
connectDB().then(() => {
  app.listen(PORT, () => {
    console.log("Server running on http://localhost:" + PORT);
  });
});
```

IMPORTANT

dotenv.config() must be the first line. db.js reads process.env.MONGO_URI, so the .env values must be loaded before anything else runs.

STEP 11 Add run scripts

WHAT	Edit package.json so **main** points to src/server.js and add the **start** and **dev** scripts.
WHERE	**package.json** in the project root.
WHY	Scripts give short commands: **npm run dev** for development (auto restart), **npm start** for normal run.
WHEN	After all code files exist.

package.json

```
{
  "name": "student-management",
  "version": "1.0.0",
  "description": "Student Management REST API (Node.js + Express + MongoDB)",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "license": "ISC"
}
```

DO NOT DELETE

Your real package.json also has a **dependencies** and a **devDependencies** section created by npm install. **Keep them**, just add or edit main and scripts.

STEP 12 Run the project

WHAT	Start MongoDB, then start the API.
WHERE	Terminal, inside the project folder.
WHY	To see that everything is connected and the server is alive.
WHEN	After all files are saved.

Terminal

```
# 1) Make sure MongoDB is running (local install)
#    Windows: it runs as a service.   Mac: brew services start mongodb-community
#    Linux:   sudo systemctl start mongod
#    (Skip this if you use Atlas.)

# 2) Start the API in development mode
npm run dev
```

You should see these lines:

Expected output

```
MongoDB connected: 127.0.0.1
Server running on http://localhost:5000
```

Open **http://localhost:5000/** in a browser. You should see the message `Student Management API is running`.

STEP 13 Test all CRUD APIs

WHAT	Send requests to every endpoint and check the responses.
WHERE	A second terminal (curl) or **Postman**. In Postman choose **Body > raw > JSON** for POST and PUT.
WHY	To prove that Create, Read, Update, and Delete all work, and to see how errors are returned.
WHEN	After the server runs. Test in this order, because later tests need the id from the first one.

13.1 CREATE: POST /api/students

Request

```
curl -X POST http://localhost:5000/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Rahul Sharma","email":"rahul@example.com",
       "age":20,"course":"B.Tech CSE","phone":"9876543210"}'
```

Response (201 Created)

```
{
  "success": true,
  "message": "Student created",
  "data": {
    "_id": "665f1c2e8b3a4d0012ab34cd",
    "name": "Rahul Sharma",
    "email": "rahul@example.com",
    "age": 20,
    "course": "B.Tech CSE",
    "phone": "9876543210",
    "createdAt": "2026-11-09T10:15:30.000Z",
    "updatedAt": "2026-11-09T10:15:30.000Z"
  }
}
```

Copy the **_id** value. You need it for the next tests (the examples use `665f1c2e8b3a4d0012ab34cd`; use your own).

13.2 READ ALL: GET /api/students

Request

```
curl http://localhost:5000/api/students
```

Response (200 OK)

```
{
  "success": true,
  "count": 1,
  "data": [ { "_id": "665f1c2e8b3a4d0012ab34cd", "name": "Rahul Sharma", "...": "..." } ]
}
```

13.3 READ ONE: GET /api/students/:id

Request

```
curl http://localhost:5000/api/students/665f1c2e8b3a4d0012ab34cd
```

Returns `{ "success": true, "data": { ... } }` with status 200.

13.4 UPDATE: PUT /api/students/:id

Request

```
curl -X PUT http://localhost:5000/api/students/665f1c2e8b3a4d0012ab34cd \
  -H "Content-Type: application/json" \
  -d '{"course":"B.Tech AI","age":21}'
```

Send only the fields you want to change. The response has `"message": "Student updated"` and the new data.

13.5 DELETE: DELETE /api/students/:id

Request

```
curl -X DELETE http://localhost:5000/api/students/665f1c2e8b3a4d0012ab34cd
```

Response (200 OK)

```
{ "success": true, "message": "Student deleted" }
```

13.6 Error tests (very important)

Test	How	You get
Missing required field	POST without email	400 Email is required
Duplicate email	POST the same email twice	409 Email already exists
Bad id	GET /api/students/123	400 Invalid student id
Id not in database	GET with a valid-looking but unused id	404 Student not found
Age out of range	POST with age 500	400 Age must be at most 100

API summary

Method	URL	Body	Success	Errors
POST	/api/students	name, email, course (+age, phone)	201	400, 409
GET	/api/students	none	200	500
GET	/api/students/:id	none	200	400, 404
PUT	/api/students/:id	any fields to change	200	400, 404, 409
DELETE	/api/students/:id	none	200	400, 404

HTTP status codes used

Code	Meaning	When
200	OK	Read, update, delete worked
201	Created	New student saved
400	Bad Request	Validation failed or id is not valid
404	Not Found	No student with that id
409	Conflict	Email already exists
500	Server Error	Unexpected problem (for example database down)

Common problems and fixes

Problem	Reason	Fix
MongoDB connection failed (ECONNREFUSED)	MongoDB is not running	Start the mongod service, or use an Atlas URL
req.body is empty / undefined	Missing header or wrong body type	Send JSON and add Content-Type: application/json
Cannot find module '../models/...'	Wrong file name or path	Check spelling, such as student.model.js
MONGO_URI undefined	.env missing or not loaded	Keep .env in the root and dotenv.config() first in server.js
Port 5000 already in use	Another app uses the port	Change PORT in .env
E11000 duplicate key	Same email already saved	Use another email (this returns 409)

What to learn next

- **Pagination and search**: use `?page=1&limit=10` and filter with `find({ course })`.
- **Middleware** (when you are ready): authentication with JWT, a central error handler, request logging.
- **Input validation library** such as Joi or express-validator for more detailed rules.

- **More resources**: teachers, courses, attendance, with the same folder pattern (model, controller, routes).
- **Deployment**: MongoDB Atlas for the database and Render or Railway for the API.

> **GOLDEN RULE**
> Pattern to remember for every new resource: **model** (what it looks like) **controller** (what it does) **routes** (how to reach it) plug into **app.js**.